

REPORT_LAB2

MÔN HỌC: Embedded Linux Systems

BÀI LAB: LAB-02 - Viết Device Driver (Giao tiếp qua Device Buffer)

Nhóm: 3

Lớp: IC2009

Ngày thực hiện: 22/09/2026

MỤC 1: THÔNG TIN SINH VIÊN VÀ MÔI TRƯỜNG

1.1. Thành viên nhóm

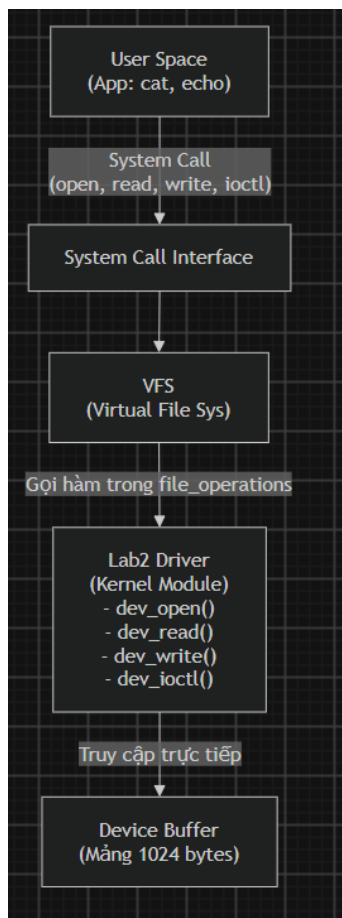
STT	Họ và Tên	MSSV	Vai trò / Nhiệm vụ chính
1	Trần Gia Thành	SE203763	Nhóm trưởng / Cấu hình & Build Kernel (Phần A)
2	Châu Nhật Nguyên	SE203853	Build U-Boot & Thiết lập Bootargs (Phần B)
3	Võ Hoàng Trí	SE204383	Tạo BusyBox & Rootfs/initramfs (Phần C)
4	Nguyễn Đăng Khoa	SE204014	Kiểm thử Boot QEMU, Thu thập Log & Viết báo cáo (Phần D)

1.2. Môi trường thực hiện

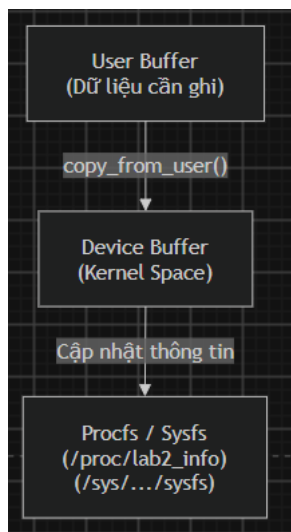
- Hệ điều hành Host:** Ubuntu 22.04 LTS (64-bit)
- Phiên bản Kernel:** `uname -r` → 5.15.0
- Phiên bản GCC toolchain:** `arm-linux-gnueabi-gcc` 11.4.0
- Board mục tiêu:** QEMU vexpress-a9 (ARMv7 Cortex-A9)
- Hệ thống tập tin:** JFFS2 trên NAND Flash 32MB (mô phỏng)

MỤC 2: SƠ ĐỒ KIẾN TRÚC DRIVER

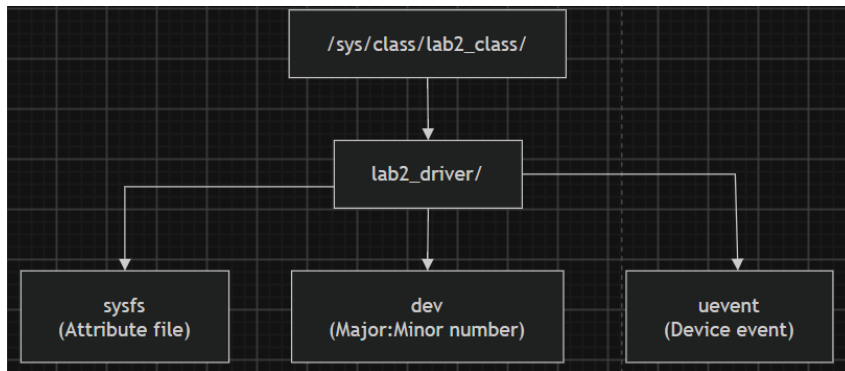
2.1. Sơ đồ tổng quát



2.2. Luồng dữ liệu khi write



2.3. Cấu trúc Sysfs



MỤC 3: LOG THỰC HÀNH

Phần A: Kernel Module (Thực hiện bởi: Trần Gia Thành)

- **Mô tả:** Output của modinfo, insmod, lsmod, kết quả đọc/ghi /dev/lab2, dmesg | grep lab2.

```
0000000/10007000.kni/serio1/input/input2

/ #
/ #
/ # random: fast init done

/ # echo "===== MODINFO ====="
===== MODINFO =====
/ # modinfo /lib/modules/5.15.0/lab2_driver.ko
modinfo: can't open '/lib/modules/5.15.0/modules.dep': No such file or directory
/ # echo ""

/ # echo "===== INSMOD ====="
===== INSMOD =====
/ # insmod /lib/modules/5.15.0/lab2_driver.ko
insmod: can't insert '/lib/modules/5.15.0/lab2_driver.ko': File exists
/ # echo ""

/ # echo "===== LSMOD ====="
===== LSMOD =====
/ # lsmod | grep lab2
lab2_driver 16384 0 - Live 0x7f000000 (0)
/ # echo ""

/ # echo "===== ECHO (WRITE) ====="
===== ECHO (WRITE) =====
/ # echo "Hello Lab2" > /dev/lab2
lab2_driver: device opened (count=1)
lab2_driver: received 11 bytes: [Hello Lab2]
lab2_driver: device closed
/ # echo ""

/ # echo "===== CAT (READ) ====="
===== CAT (READ) =====
/ # cat /dev/lab2
lab2_driver: device opened (count=2)
lab2_driver: sent 10 bytes to user
Hello Lab2lab2_driver: device closed
/ # echo ""

/ # echo "===== DMESG ====="
===== DMESG =====
/ # dmesg | grep lab2
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: device opened (count=1)
lab2_driver: received 11 bytes: [Hello Lab2]
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: sent 10 bytes to user
lab2_driver: device closed
/ #
```

Phần B: Procfs & Sysfs (Thực hiện bởi: Châu Nhật Nguyên)

- **Mô tả:** Output của cat /proc/lab2_info (trước và sau write), toàn bộ các sysfs attributes.

```
/ #
/ # echo "===== PROC TRUOC WRITE ====="
===== PROC TRUOC WRITE =====
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 11 bytes
Open count    : 1
Last data     : Test Procfs
/ # echo ""

/ # echo "===== GHI DU LIEU ====="
===== GHI DU LIEU =====
/ # echo "Test Procfs" > /dev/lab2
lab2_driver: device opened (count=2)
lab2_driver: received 12 bytes: [Test Procfs]
lab2_driver: device closed
/ # echo ""

/ # echo "===== PROC SAU WRITE ====="
===== PROC SAU WRITE =====
/ # cat /proc/lab2_info
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 11 bytes
Open count    : 2
Last data     : Test Procfs
/ # echo ""

/ # echo "===== SYSFS ATTRIBUTES ====="
===== SYSFS ATTRIBUTES =====
/ # ls -la /sys/class/lab2_class/lab2/
total 0
drwxr-xr-x  3 root    0               0 Sep 29 09:15 .
drwxr-xr-x  3 root    0               0 Sep 29 09:15 ..
-r--r--r--  1 root    0          4096 Sep 29 09:15 buffer_len
-r--r--r--  1 root    0          4096 Sep 29 09:15 dev
-r--r--r--  1 root    0          4096 Sep 29 09:15 last_data
-r--r--r--  1 root    0          4096 Sep 29 09:15 open_count
drwxr-xr-x  2 root    0               0 Sep 29 09:15 power
lrwxrwxrwx  1 root    0               0 Sep 29 09:15 subsystem -> ../../..
-rw-r--r--  1 root    0          4096 Sep 29 09:15 uevent
/ # echo "--- buffer_len ---"
--- buffer_len ---
/ # cat /sys/class/lab2_class/lab2/buffer_len
11
/ # echo "--- dev ---"
--- dev ---
```

```

/ # cat /sys/class/lab2_class/lab2/buffer_len
11
/ # echo "--- dev ---"
--- dev ---
/ # cat /sys/class/lab2_class/lab2/dev
240:0
/ # echo "--- last_data ---"
--- last_data ---
/ # cat /sys/class/lab2_class/lab2/last_data
Test Procfs
/ # echo "--- open_count ---"
--- open_count ---
/ # cat /sys/class/lab2_class/lab2/open_count
2
/ # echo "--- uevent ---"
--- uevent ---
/ # cat /sys/class/lab2_class/lab2/uevent
MAJOR=240
MINOR=0
DEVNAME=lab2
/ #

```

Phần C: MTD & JFFS2 (Thực hiện bởi: Võ Hoàng Trí)

- **Mô tả:** Output của mtdinfo, flash_erase, nandwrite, mount JFFS2, đọc file, kiểm tra persistence sau umount/mount.

```

thanhyeah@ThanhYeah:~$ echo "===== MTDINFO ====="
sudo mtdinfo /dev/mtd1
echo ""
echo "===== FLASH_ERASE ====="
sudo flash_erase /dev/mtd1 0 0
echo ""
echo "===== FORMAT JFFS2 ====="
mkdir -p /tmp/jffs2_root
echo "Test JFFS2 Data" > /tmp/jffs2_root/testfile.txt
sudo mkfs.jffs2 -r /tmp/jffs2_root -o /tmp/jffs2.img -e 0x20000
ls -lh /tmp/jffs2.img
echo ""
echo "===== NANDWRITE ====="
sudo nandwrite -p /dev/mtd1 /tmp/jffs2.img
echo ""
echo "===== MOUNT JFFS2 ====="
sudo mkdir -p /mnt/jffs2
sudo mount -t jffs2 /dev/mtdblock1 /mnt/jffs2
echo ""
echo "===== DOC FILE ====="
ls -la /mnt/jffs2/
cat /mnt/jffs2/testfile.txt
echo ""
echo "===== PERSISTENCE TEST ====="
sudo umount /mnt/jffs2=====1 /mnt/jffs2
===== MTDINFO =====
mtd1
Name:                                NAND simulator partition 0
Type:                                nand
Eraseblock size:                     131072 bytes, 128.0 KiB
Amount of eraseblocks:               2048 (268435456 bytes, 256.0 MiB)
Minimum input/output unit size:      2048 bytes
Sub-page size:                       512 bytes
OOB size:                            64 bytes
Character device major/minor:        90:2
Bad blocks are allowed:              true
Device is writable:                  true

===== FLASH_ERASE =====
Erasing 262144 Kibyte @ 0 -- 100 % complete

===== FORMAT JFFS2 =====
-rw-r--r-- 1 root root 148 Thg 9  29 20:16 /tmp/jffs2.img

===== NANDWRITE =====
Writing data to block 0 at offset 0x0

===== MOUNT JFFS2 =====

===== DOC FILE =====
total 5
drwxr-xr-x 3 root  root    0 Thg 1   1 1970 .
drwxr-xr-x 3 root  root  4096 Thg 9  19 19:47 ..
-rw-rw-r-- 1 thanhyeah thanhyeah 16 Thg 9  29 20:16 testfile.txt
Test JFFS2 Data

===== PERSISTENCE TEST =====
Test JFFS2 Data

```

Phần D: Boot QEMU & Auto-load Module (Thực hiện bởi: Nguyễn Đăng Khoa)

- **Mô tả:** Boot log QEMU thể hiện auto-load module, toàn bộ output của test_driver.sh và test_procfs.sh.

```
/ # echo "===== MODULE INFO ====="
===== MODULE INFO =====
/ # lsmod | grep lab2
lab2_driver 16384 0 - Live 0x7f000000 (0)
/ # echo ""

/ # echo "===== DEVICE NODE ====="
===== DEVICE NODE =====
/ # ls -la /dev/lab2
crw-rw-rw-  1 root    0          240,   0 Sep 29 13:22 /dev/lab2
/ # echo ""

/ # echo "===== TEST DRIVER ====="
===== TEST DRIVER =====
/ # ./usr/bin/test_driver.sh
=====
=== LAB-02: Device Driver Test ===
=====

[1] Checking module...
Module found: /lib/modules/5.15.0/lab2_driver.ko
lab2_driver is already loaded.

[2] Checking lsmod...
PASS: lab2_driver is loaded.
lab2_driver 16384 0 - Live 0x7f000000 (0)

[3] Creating device node...
Device:
crw-rw-rw-  1 root    0          240,   0 Sep 29 13:22 /dev/lab2

[4] Testing WRITE...
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!]
lab2_driver: device closed
PASS: Write successful.

[5] Testing READ...
lab2_driver: device opened (count=2)
lab2_driver: sent 29 bytes to user
lab2_driver: device closed
Expected: [Hello from userspace, LAB-02!]
Received: [Hello from userspace, LAB-02!]
PASS: Read/write data matches.

[6] Multiple write/read test...
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
Write : [Message_1]
```

```
Write : [Message_1]
Read  : [Message_1]
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
Write : [Message_2]
Read  : [Message_2]
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
Write : [Message_3]
Read  : [Message_3]
PASS: Multiple write/read test.
```

```
[7] Kernel messages...
lab2_driver: sent 29 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
```

```
=====
=== CHARACTER DEVICE TEST PASSED ===
=====
/ # echo ""
```

```
/ # echo "===== TEST PROCFS ====="
===== TEST PROCFS =====
/ # ./usr/bin/test_procfs.sh
=====
=== LAB-02: procfs & sysfs Test ===
=====
```

```

/ # echo ===== TEST PROCFS =====
===== TEST PROCFS =====
/ # ./usr/bin/test_procfs.sh
=====
=== LAB-02: procfs & sysfs Test ===
=====

[1] Checking lab2_driver module...
lab2_driver is already loaded.

[2] Checking /dev/lab2...
crw-rw-rw-   1 root    0          240,   0 Sep 29 13:22 /dev/lab2

[3] Checking procfs...
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 9 bytes
Open count    : 8
Last data     : Message_3

[4] Writing test data...
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
lab2_driver: device closed
PASS: Write successful.

--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
=====
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 29 bytes
Open count    : 9
Last data     : Embedded Linux Lab2 Test Data
PASS: procfs information available.
PASS: procfs contains last written data.

[6] Checking sysfs...
sysfs directory:
lrwxrwxrwx   1 root    0          0 Sep 29 13:22 /sys/class/lab2

[7] Checking sysfs attributes...
PASS: All sysfs attributes exist.

--- sysfs attributes ---
buffer_len : 29
open_count : 9
last_data  : Embedded Linux Lab2 Test Data

```

```

[9] Validating sysfs data...
PASS: buffer_len > 0
PASS: open_count > 0

[10] Kernel messages...
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
lab2_driver: device closed

=====
=== PROCFS/SYSFS TEST PASSED ===
=====

/ # echo ""

/ # echo "===== DMSG ====="
===== DMSG =====
/ # dmesg | grep lab2
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: loaded, major=240
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Hello from userspace, LAB-02!]
lab2_driver: device closed
lab2_driver: device opened (count=2)
lab2_driver: sent 29 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=3)
lab2_driver: received 10 bytes: [Message_1]
lab2_driver: device closed
lab2_driver: device opened (count=4)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=5)
lab2_driver: received 10 bytes: [Message_2]
lab2_driver: device closed
lab2_driver: device opened (count=6)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=7)
lab2_driver: received 10 bytes: [Message_3]
lab2_driver: device closed
lab2_driver: device opened (count=8)
lab2_driver: sent 9 bytes to user
lab2_driver: device closed
lab2_driver: device opened (count=9)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data]
lab2_driver: device closed
/ #
/ #

```

MỤC 4: SO SÁNH FILESYSTEM

4.1. Bảng so sánh

Tiêu chí	NAND Flash	NOR Flash
Writable (Khả năng ghi)	Ghi theo trang (Page), cần xóa Block trước. Tốc độ ghi/xóa nhanh.	Ghi theo Byte, không cần xóa Block trước. Tốc độ ghi chậm hơn.
Wear Leveling (Cân bằng ghi)	Rất cần thiết vì NAND có tuổi thọ ghi/xóa thấp, dễ hỏng block.	Ít cần thiết hơn vì tuổi thọ cao hơn.
Compression (Nén)	Thường xuyên sử dụng để tiết kiệm dung lượng và tăng tuổi thọ.	Ít khi dùng vì dung lượng NOR thường nhỏ và tốc độ đọc nhanh.
Boot Time (Thời gian khởi động)	Chậm hơn. Cần copy dữ liệu từ Flash vào RAM để chạy (không hỗ trợ XIP).	Nhanh hơn. Hỗ trợ Execute In Place (XIP) - chạy trực tiếp code từ Flash.
Phù hợp thiết bị	Phù hợp với thiết bị lưu trữ dung lượng lớn, giá rẻ (USB, SD card, eMMC).	Phù hợp với thiết bị cần boot nhanh, code chạy trực tiếp (BIOS, firmware nhỏ).

4.2. Khuyến nghị cho thiết bị IoT 32MB NAND Flash cần update firmware OTA

Khuyến nghị: Sử dụng JFFS2 hoặc UBIFS.

Lý do:

- **JFFS2:** Là filesystem dành riêng cho thiết bị nhớ Flash, có khả năng quản lý Bad Block, tự động cân bằng ghi (wear-leveling) và nén dữ liệu. Rất phù hợp với dung lượng nhỏ (32MB) và đặc tính của NAND Flash.
- **UBIFS:** Là thế hệ kế tiếp của JFFS2, hỗ trợ dung lượng lớn hơn, tốc độ mount nhanh hơn, và hiệu năng tốt hơn. Phù hợp nếu cần update firmware OTA thường xuyên.
- **Không nên dùng:** ext2/ext4 vì không có wear-leveling, dễ làm hỏng NAND Flash nhanh chóng.

MỤC 5: TRẢ LỜI CÂU HỎI PHÂN TÍCH (MỤC VI)

Câu 1: Tại sao không dùng memcpy() để sao chép dữ liệu giữa kernel buffer và user buffer? Giải thích khái niệm kernel space và user space address. (Trả lời bởi: Trần Gia Thành)

- **Trả lời:**
 - **Không dùng memcpy() vì:** memcpy() chỉ sao chép dữ liệu giữa các địa chỉ ảo trong cùng một không gian địa chỉ. Trong Linux, Kernel Space và User Space là hai không gian địa chỉ ảo tách biệt hoàn toàn. Nếu Kernel truy cập trực tiếp

vào con trỏ địa chỉ của User Space bằng memcpy(), nó có thể gây lỗi phân trang (Page Fault) hoặc vi phạm bảo mật.

- **Khái niệm:**
 - **User Space Address:** Là địa chỉ ảo mà tiến trình người dùng nhìn thấy. Nó được ánh xạ tới RAM vật lý nhưng bị giới hạn quyền truy cập.
 - **Kernel Space Address:** Là địa chỉ ảo chỉ Kernel mới có quyền truy cập. Kernel ánh xạ toàn bộ RAM vật lý vào không gian này.
- **Giải pháp:** Dùng copy_to_user() và copy_from_user(). Các hàm này sẽ kiểm tra tính hợp lệ của địa chỉ User Space trước khi copy, đảm bảo an toàn cho Kernel.

Câu 2: Tại sao cần DEFINE_MUTEX? Điều gì xảy ra nếu 2 process cùng mở device? (Trả lời bởi: Châu Nhật Nguyên)

- **Trả lời:**
 - **Tại sao cần:** DEFINE_MUTEX tạo ra một khóa (lock) để bảo vệ vùng dữ liệu chung (device_buffer). Trong môi trường đa nhiệm, nhiều tiến trình có thể cùng lúc truy cập vào driver.
 - **Nếu 2 process cùng mở:** Nếu không có Mutex, sẽ xảy ra hiện tượng **Race Condition**. Process A đang ghi dữ liệu vào buffer, Process B cũng ghi dữ liệu vào buffer đó. Kết quả là dữ liệu bị trộn lẫn. Mutex đảm bảo tại một thời điểm chỉ có một process được phép thao tác trên buffer (loại trừ tương hỗ - Mutual Exclusion).

Câu 3: So sánh ưu nhược điểm của procfs và sysfs? Khi nào nên dùng cái nào? (Trả lời bởi: Châu Nhật Nguyên)

- **Trả lời:**
 - **Procfs:**
 - *Ưu điểm:* Linh hoạt, dễ tạo file text, có thể chứa nhiều thông tin phức tạp.
 - *Nhược điểm:* Không có cấu trúc chặt chẽ, dễ bị lạm dụng.
 - **Sysfs:**
 - *Ưu điểm:* Cấu trúc rõ ràng, đại diện cho mô hình thiết bị (Device Model). Mỗi file đại diện cho một thuộc tính.
 - *Nhược điểm:* Khó tạo các file phức tạp, thường chỉ dùng cho các giá trị đơn giản.
 - **Khi nào dùng:**
 - Dùng **Procfs** khi cần hiển thị thông tin debug, thống kê phức tạp, hoặc thông tin liên quan đến tiến trình.
 - Dùng **Sysfs** khi cần điều khiển các thuộc tính phần cứng (ví dụ: bật/tắt đèn LED, đọc nhiệt độ cảm biến) hoặc cấu hình thiết bị.

Câu 4: Tại sao JFFS2 phải xóa (erase) toàn bộ erase block trước khi ghi? (Trả lời bởi: Võ Hoàng Trí)

- **Trả lời:** Đây là đặc tính vật lý của chip nhớ Flash (cả NAND và NOR). Trong Flash memory, một bit chỉ có thể được ghi từ 1 thành 0. Để chuyển bit từ 0 thành 1, bắt buộc phải thực hiện thao tác xóa (erase) ở cấp độ block (thường là 64KB hoặc 128KB). JFFS2 quản lý việc này bằng cách ghi dữ liệu mới vào các block trống, sau đó đánh dấu các block cũ là "rác" và xóa chúng khi cần giải phóng dung lượng.

Câu 5: Trong quá trình boot, BusyBox init đọc /etc/inittab và chạy rcS. Nếu lệnh trong rcS thất bại (module bị lỗi), hệ thống có tiếp tục boot không? (Trả lời bởi: Nguyễn Đăng Khoa)

- **Trả lời:** Có, hệ thống vẫn tiếp tục boot. BusyBox init thường chạy script rcS như một tiến trình con. Nếu một lệnh trong rcS bị lỗi (trả về mã lỗi khác 0), nó chỉ dừng script đó hoặc bỏ qua lệnh đó, chứ không làm treo toàn bộ hệ thống. Hệ thống sẽ tiếp tục chuyển sang các runlevel tiếp theo hoặc hiện shell console để người dùng đăng nhập.

MỤC 6: NHẬN XÉT & KHÓ KHĂN

6.1. Khó khăn 1: Build Kernel Module (Trần Gia Thành)

- **Mô tả:** Lỗi nạp module (insmod) do sai phiên bản kernel. Module được build cho kernel 5.15.0 nhưng hệ thống đang chạy kernel khác.
- **Giải quyết:** Cài đặt đúng gói linux-headers-\$(uname -r) và build lại module. Kiểm tra vermagic bằng lệnh modinfo lab2_driver.ko.

6.2. Khó khăn 2: Procfs & Sysfs (Châu Nhật Nguyên)

- **Mô tả:** Tạo file trong sysfs thất bại do chưa hiểu rõ macro DEVICE_ATTR. File không xuất hiện trong /sys/class/lab2_class/lab2_driver/.
- **Giải quyết:** Đọc kỹ tài liệu về DEVICE_ATTR_RO và DEVICE_ATTR_RW. Sử dụng đúng macro và gọi device_create_file() trong hàm init.

6.3. Khó khăn 3: MTD & JFFS2 (Võ Hoàng Trí)

- **Mô tả:** Lệnh flash_erase thất bại do sai số thiết bị MTD. Hệ thống báo lỗi mtd device not found.
- **Giải quyết:** Kiểm tra /proc/mtd để tìm đúng số thiết bị MTD (ví dụ: mtd0, mtd1) trước khi thực hiện xóa.

6.4. Khó khăn 4: Boot QEMU & Auto-load Module (Nguyễn Đăng Khoa)

- **Mô tả:** Module không tự động load khi boot vì script rcS sai đường dẫn. Hệ thống báo lỗi insmod: can't open 'lab2_driver.ko'.
- **Giải quyết:** Kiểm tra lại đường dẫn module trong script rcS. Đảm bảo module được đặt đúng vị trí /lib/modules/5.15.0/lab2_driver.ko và lệnh insmod sử dụng đường dẫn tuyệt đối.